a. The two genAi used are Claude and Copilot

b. Both were accessed on the web

c. Change this file to do the following

Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.

• Each solutionN.txt file should include the following:

o If no solutions are found, a single line: “No solution found.”

o If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.

o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.

Do not remove credits for the original author.

d. Claudes code:

```
/*
* Program: EECS 348 Assignment 4
* Description: C++ program that solves Sudoku puzzles using objects and
*          recursive depth-first search with backtracking.
*          Each solution must follow the row, column, and 3x3 block rules.
*          The program finds all solutions for each puzzle and writes
*          them to a matching solution file.
* Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt
*       read from the current working directory.
*       Digits 1 through 9 represent starting clues, and underscores
*       represent empty cells.
* Outputs: Writes solution1.txt through solution5.txt to the current working
*        directory, where N in solutionN.txt matches the puzzle number.
*        Each solution file contains either the single line
*        "No solution found." or every solution (in the same format as the
*        puzzle files) followed by the single line "Solutions found: n".
*        Displays each puzzle filename, the original puzzle, and a short
*        summary on the terminal.
```

```
*       Displays an error if a puzzle file cannot be opened, contains
*       invalid characters, has an incorrect cell count, cannot be read
*       completely, or if the solution file cannot be written.
* Usage: ./Claude
* Collaborators: None
* Other Sources: Claude (original C++ code selected as the basis for this assignment as
well as 2nd revisions),
*           Gemini (second GenAI program used for comparison),
*           ChatGPT (assistance with prompting, analysis, testing,
*           commenting, revisions, and source attribution)
* Author: Nathan Lenahan
* Creation Date: October 14, 2026
* Revision Date: October 15, 2026
* Revisions:
* - Added detailed comments and source attribution to the original Claude-generated
code.
* - Revised load() to reject invalid characters, require exactly 81 cells,
*   and check the entire input file while allowing whitespace.
* - Removed the unused get() method to simplify the Board class.
* - Replaced stored solution boards with immediate printing and a solution counter.
* - Added printSolution() to keep solution formatting separate from recursive search
steps.
* - Revised run() to report the solution count after the search finishes.
* - Removed the vector include because completed solutions are no longer stored.
* - Revised findEmpty() and search() to resume after each filled cell,
*   avoiding repeated scans of earlier positions during recursion.
* - Added specific loading-error descriptions for opening failures, invalid
*   characters, extra content, incomplete puzzles, and file-reading failures.
* - Added printPlain() so solutions are written in the same format as the
*   puzzle files (nine rows of nine characters, no dividers).
* - Revised printSolution() to write unlabeled solutions to the solution file.
* - Revised PuzzleRunner to take the puzzle number, create solutionN.txt, and
*   write either "No solution found." or "Solutions found: n" to it.
* - Revised the terminal output to show a summary instead of every solution.
*/

// The following include statements and namespace declaration are from the original
code generated by Claude
#include <iostream> // Provides terminal output using cout and the ostream type
#include <fstream> // Provides file input using ifstream and file output using ofstream
#include <string> // Provides the string type and the to_string conversion
#include <cctype> // Provides isspace for checking whether a character is whitespace
```

```
using namespace std; // Allows standard library names to be used without the std:: prefix

// The following Board class is based on the original code generated by Claude
// Its load and findEmpty methods were revised and its unused get method was removed with help from ChatGPT
// Its printPlain method was added to write solutions in the puzzle file format
// It stores the Sudoku grid and handles loading, printing, and checking placements
class Board { // Starts the Board class
public: // Makes the following constants, constructor, and methods accessible outside the class

   static const int SIZE = 9; // Defines the shared constant for the number of rows and columns
   static const int BLOCK = 3; // Defines the shared constant for the number of rows and columns in each block

   // The following Board constructor is from the original code generated by Claude
   // It initializes every cell to 0 so a new board starts empty
   Board() { // Starts the default Board constructor

      for (int r = 0; r < SIZE; r++) // Loops through all nine rows
         for (int c = 0; c < SIZE; c++) // Loops through all nine columns in the current row
            cells[r][c] = 0; // Sets the current cell to 0 to represent an empty cell

   } // Ends the Board constructor

   // The following load method is based on the original code generated by Claude
   // It was revised with help from ChatGPT to validate all cells and describe loading failures
   // It accepts exactly 81 digits or underscores while allowing whitespace
   bool load(const string& filename, string& errorMessage) { // Loads the puzzle and provides an error description through a reference

      errorMessage.clear(); // Removes any error description left from an earlier call

      ifstream in(filename.c_str()); // Attempts to open the puzzle file

      if (!in) { // Checks whether the file could not be opened
         errorMessage = "Could not open the file."; // Reports the failure without guessing its underlying cause
         return false; // Stops loading because the file is unavailable
      } // Ends the file-opening check
```

```
    int count = 0; // Tracks the number of accepted cells
    char ch; // Stores one character read from the file

    while (in.get(ch)) { // Reads characters until the file ends or reading fails

      if (isspace(static_cast<unsigned char>(ch))) { // Checks whether the character is whitespace
        continue; // Skips whitespace without counting it as a cell
      } // Ends the whitespace check

      if (count >= SIZE * SIZE) { // Checks for non-whitespace content after all 81 cells
        errorMessage = "Extra content found after 81 cells."; // Explains that the file contains too much cell data
        return false; // Rejects the extra content before accessing the board
      } // Ends the extra-content check

      int r = count / SIZE; // Calculates the row index for the next cell
      int c = count % SIZE; // Calculates the column index for the next cell

      if (ch >= '1' && ch <= '9') { // Checks whether the character is an allowed digit
        cells[r][c] = ch - '0'; // Converts and stores the digit as an integer

      } else if (ch == '_') { // Checks whether the character represents a blank
        cells[r][c] = 0; // Stores 0 to represent the empty cell

      } else { // Handles a character that is not an allowed digit or underscore
        errorMessage = "Invalid character at cell " + to_string(count + 1)
          + ". Expected a digit from 1 through 9 or an underscore."; // Identifies the cell and explains the allowed characters
        return false; // Rejects the invalid input

      } // Ends the cell-character validation

      count++; // Increases the number of accepted cells

    } // Ends the file-reading loop

    if (in.bad() || !in.eof()) { // Checks for a reading failure rather than a normal end of the file
      errorMessage = "A file-reading error occurred; the exact cause is unknown."; // Reports the stream failure without inventing a cause
      return false; // Rejects a file that could not be read normally
    } // Ends the reading-error check
```

```
    if (count != SIZE * SIZE) { // Checks whether the file ended before all 81 cells were read
        errorMessage = "Expected 81 cells, but found " + to_string(count) + "."; // Reports the actual number of accepted cells
        return false; // Rejects the incomplete puzzle
    } // Ends the cell-count check

    return true; // Reports successful loading after every validation check passes

  } // Ends the load method

  // The following set method is from the original code generated by Claude
  // It updates the value stored at a given row and column
  void set(int r, int c, int v) { // Starts the method that changes one cell
    cells[r][c] = v; // Stores the provided value in the specified cell
  } // Ends the set method

  // The following findEmpty method is based on the original code generated by Claude
  // It was revised with help from ChatGPT to begin searching at a provided position
  // This avoids repeatedly checking earlier cells that are already filled
  bool findEmpty(int& row, int& col, int startIndex = 0) const { // Searches for an empty cell starting at the provided index

    for (int position = startIndex; position < SIZE * SIZE; position++) { // Checks positions from the starting index through the last cell

      int r = position / SIZE; // Converts the position into a row index
      int c = position % SIZE; // Converts the position into a column index

      if (cells[r][c] == 0) { // Checks whether the current cell is empty

        row = r; // Provides the empty cell's row to the caller
        col = c; // Provides the empty cell's column to the caller
        return true; // Reports that an empty cell was found

      } // Ends the empty-cell check

    } // Ends the position-search loop

    return false; // Reports that no empty cells were found at or after the starting index

  } // Ends the findEmpty method

  // The following canPlace method is from the original code generated by Claude
```

```
  // It checks whether a digit already appears in the selected row, column, or block
  bool canPlace(int row, int col, int v) const { // Starts checking whether value v can be placed at the given position

    for (int i = 0; i < SIZE; i++) { // Loops through all nine positions in the selected row and column
      if (cells[row][i] == v) return false; // Rejects the value if it already appears in the row
      if (cells[i][col] == v) return false; // Rejects the value if it already appears in the column
    } // Ends the row and column checks

    int br = (row / BLOCK) * BLOCK; // Calculates the starting row of the cell's 3x3 block
    int bc = (col / BLOCK) * BLOCK; // Calculates the starting column of the cell's 3x3 block

    for (int r = br; r < br + BLOCK; r++) // Loops through the three rows of the block
      for (int c = bc; c < bc + BLOCK; c++) // Loops through the three columns of the block
        if (cells[r][c] == v) return false; // Rejects the value if it already appears in the block
    return true; // Reports that the value has no row, column, or block conflicts
  } // Ends the canPlace method

  // The following givensAreConsistent method is from the original code generated by Claude
  // It checks each starting clue for conflicts while preserving the original board
  bool givensAreConsistent() { // Starts checking whether the starting clues follow the Sudoku rules

    for (int r = 0; r < SIZE; r++) // Loops through every row
      for (int c = 0; c < SIZE; c++) { // Loops through every column in the current row

        int v = cells[r][c]; // Saves the value currently stored in the cell

        if (v == 0) continue; // Skips empty cells because they contain no clue to check

        cells[r][c] = 0; // Temporarily removes the clue so it does not conflict with itself
        bool ok = canPlace(r, c, v); // Checks the saved clue against the other digits in its row, column, and block
        cells[r][c] = v; // Restores the clue before continuing or returning

        if (!ok) return false; // Rejects the board if the clue conflicts with another digit
      } // Ends the column loop
    return true; // Reports that all starting clues passed the conflict checks
  } // Ends the givensAreConsistent method
```

```
  // The following print method is from the original code generated by Claude
  // It displays the board with underscores for blanks and dividers between blocks
  void print(ostream& out) const { // Starts printing to the provided output stream without changing the board

    for (int r = 0; r < SIZE; r++) { // Loops through each row of the board
      if (r > 0 && r % BLOCK == 0) out << "------+-------+------\n"; // Prints a horizontal divider before row indexes 3 and 6
      for (int c = 0; c < SIZE; c++) { // Loops through each column in the current row
        if (c > 0 && c % BLOCK == 0) out << "| "; // Prints a vertical divider before column indexes 3 and 6
        if (cells[r][c] == 0) out << "_"; // Displays an underscore for an empty cell
        else          out << cells[r][c]; // Displays the digit stored in a filled cell
        out << (c < SIZE - 1 ? " " : ""); // Adds a space after each cell except the last cell in the row
      } // Ends the column loop

      out << "\n"; // Moves to the next line after printing the row

    } // Ends the row loop
  } // Ends the print method

  // The following printPlain method was added with help from ChatGPT
  // It writes the board in the same format as the puzzle files: nine rows of nine characters
  // It uses no spaces or dividers, so the output can be read back by load()
  void printPlain(ostream& out) const { // Starts writing to the provided output stream without changing the board

    for (int r = 0; r < SIZE; r++) { // Loops through each row of the board

      for (int c = 0; c < SIZE; c++) { // Loops through each column in the current row

        if (cells[r][c] == 0) out << '_'; // Writes an underscore for an empty cell
        else          out << cells[r][c]; // Writes the digit stored in a filled cell

      } // Ends the column loop

      out << "\n"; // Moves to the next line after writing the row

    } // Ends the row loop
  } // Ends the printPlain method
```

```
private: // Prevents code outside Board from directly accessing the following data
   int cells[SIZE][SIZE]; // Stores the 9x9 grid as integers, with 0 representing an empty cell
}; // Ends the Board class

// The following SudokuSolver class is based on the original code generated by Claude
// Solution handling and search positioning were revised with help from ChatGPT
// This class finds all solutions using recursive backtracking
class SudokuSolver { // Starts the SudokuSolver class
public: // Makes the constructor and solveAll method accessible outside the class

   // The following constructor is based on the original code generated by Claude
   // It was revised with help from ChatGPT to initialize a solution counter
   explicit SudokuSolver(const Board& puzzle)
      : board(puzzle), solutionCount(0) {} // Copies the puzzle and initializes the number of solutions to 0

   // The following solveAll method is based on the original code generated by Claude
   // It was revised with help from ChatGPT to accept an output stream and return a count instead of a solution vector
   // The output stream is the solution file, which receives each solution as it is found
   unsigned long long solveAll(ostream& out) { // Starts solving and returns the number of solutions found

      solutionCount = 0; // Resets the counter before starting a new search

      if (board.givensAreConsistent()) { // Allows searching only when the starting clues have no conflicts
         search(out); // Starts searching from position 0 using the default starting index
      } // Ends the starting-clue check

      return solutionCount; // Returns the total number of solutions found

   } // Ends the solveAll method

private: // Restricts access to the working board, counter, and helper methods

   Board board; // Stores the working board modified during backtracking
   unsigned long long solutionCount; // Counts solutions without storing their completed boards

   // The following printSolution method was added with help from ChatGPT
   // It keeps solution formatting in a separate method from the recursive search
   // It writes the board in the puzzle file format followed by a blank line to separate solutions
```

```
  void printSolution(ostream& out) const { // Starts writing the current completed board

    board.printPlain(out); // Writes the completed board to the provided output stream
    out << "\n"; // Adds a blank line after the solution

  } // Ends the printSolution method

  // The following search method is based on the original code generated by Claude
  // It was revised with help from ChatGPT to print solutions immediately and resume searching after each filled cell
  // Earlier positions are already filled along the current recursive path
  void search(ostream& out, int startIndex = 0) { // Starts a recursive search using the provided output stream and cell position

    int row, col; // Receives the coordinates of the next empty cell

    if (!board.findEmpty(row, col, startIndex)) { // Checks whether no empty cells remain from the starting position onward

      solutionCount++; // Increases the count because all cells are filled along this search path
      printSolution(out); // Writes the completed board before backtracking changes it
      return; // Ends this call while allowing earlier calls to continue searching

    } // Ends the completed-board check

    int nextIndex = row * Board::SIZE + col + 1; // Calculates the position immediately after the selected cell

    for (int v = 1; v <= 9; v++) { // Tries candidate digits from 1 through 9

      if (board.canPlace(row, col, v)) { // Checks whether the candidate has no row, column, or block conflicts

        board.set(row, col, v); // Places the candidate in the selected empty cell
        search(out, nextIndex); // Recursively searches beginning after the cell just filled
        board.set(row, col, 0); // Clears the attempted placement before trying another candidate

      } // Ends the allowed-placement check

    } // Ends the candidate loop

  } // Ends the search method
```

```
}; // Ends the SudokuSolver class

// The following PuzzleRunner class is based on the original code generated by Claude
// Its constructor and run method were revised with help from ChatGPT
// It loads one puzzle file, runs the solver, and writes the results to the matching solution file
class PuzzleRunner { // Starts the PuzzleRunner class
public: // Makes the following constructor and method accessible outside the class

  // The following PuzzleRunner constructor is based on the original code generated by Claude
  // It was revised with help from ChatGPT to take the puzzle number and build both filenames
  explicit PuzzleRunner(int number)
    : filename("puzzle" + to_string(number) + ".txt"),
      solutionFilename("solution" + to_string(number) + ".txt") {} // Builds puzzleN.txt and solutionN.txt and prevents implicit conversion to PuzzleRunner

  // The following run method is based on the original code generated by Claude
  // It was revised with help from ChatGPT to display specific loading errors and write results to a solution file
  // It displays the original puzzle and writes either all solutions with a count or the no-solution message
  void run() { // Starts processing one puzzle file

    cout << "==============================\n"; // Prints the upper divider for the filename heading
    cout << "Puzzle file: " << filename << "\n"; // Displays the puzzle filename
    cout << "==============================\n"; // Prints the lower divider for the filename heading

    Board puzzle; // Creates an empty Board object for the original puzzle
    string errorMessage; // Receives the reason if loading the puzzle fails

    if (!puzzle.load(filename, errorMessage)) { // Attempts loading and checks whether it failed

      cout << "Error loading " << filename << ": " << errorMessage << "\n\n"; // Displays the filename and specific loading failure
      return; // Stops processing this file while allowing the main loop to continue

    } // Ends the loading-failure check
```

```
    cout << "Puzzle:\n"; // Displays the heading for the original puzzle
    puzzle.print(cout); // Prints the original puzzle to the terminal
    cout << "\n"; // Adds a blank line after the puzzle

    ofstream outFile(solutionFilename.c_str()); // Opens the solution file, replacing any
earlier contents

    if (!outFile) { // Checks whether the solution file could not be opened for writing

        cout << "Error writing " << solutionFilename << ": Could not open the file.\n\n"; //
Displays the solution filename and the failure
        return; // Stops processing this puzzle while allowing the main loop to continue

    } // Ends the output-opening check

    SudokuSolver solver(puzzle); // Creates a solver with its own working copy of the
puzzle
    unsigned long long solutionCount = solver.solveAll(outFile); // Writes each solution
to the file during the search and receives the total count

    if (solutionCount == 0) { // Checks whether the search found no solutions

        outFile << "No solution found.\n"; // Writes the required single no-solution line to
the file

    } else { // Handles puzzles with one or more solutions

        outFile << "Solutions found: " << solutionCount << "\n"; // Writes the required
single count line after all solutions

    } // Ends the solution-count check

    outFile.close(); // Closes the file so all buffered output is written

    if (outFile.fail()) { // Checks whether any write or close operation failed

        cout << "Error writing " << solutionFilename << ": A file-writing error occurred; the
exact cause is unknown.\n\n"; // Reports the failure without inventing a cause
        return; // Stops processing this puzzle while allowing the main loop to continue

    } // Ends the writing-error check

    if (solutionCount == 0) { // Checks whether the terminal summary should report no
solutions
```

```
        cout << "No solution found. Result written to " << solutionFilename << "\n\n"; //
Displays the no-solution summary and the output filename

      } else { // Handles puzzles with one or more solutions

        cout << "Solutions found: " << solutionCount
           << ". Results written to " << solutionFilename << "\n\n"; // Displays the solution
count and the output filename

      } // Ends the terminal summary check

   } // Ends the run method

private: // Restricts direct access to the following data members to PuzzleRunner
   string filename; // Stores the name of the puzzle file this runner processes
   string solutionFilename; // Stores the name of the solution file this runner writes

}; // Ends the PuzzleRunner class

// The following main function is based on the original code generated by Claude
// It was revised with help from ChatGPT to pass the puzzle number to PuzzleRunner
// It creates and runs a PuzzleRunner object for each of the five puzzle files
int main() { // Starts the program's main function
   for (int i = 1; i <= 5; i++) { // Loops through puzzle numbers 1 through 5
      PuzzleRunner runner(i); // Creates an object that processes puzzleN.txt and writes
solutionN.txt
      runner.run(); // Loads the puzzle, displays it, solves it, and writes the results
   } // Ends the loop that processes the five puzzle files

   return 0; // Ends the program successfully

} // Ends the main function
```

e.

- Claudes code is correct for all Sudoku inputs that meet the stated requirements. It validates input files, checks that the starting clues are consistent, and uses recursive backtracking to find all solutions.

- The worst-case time complexity is $O(9^n)$, where n is the number of blank cells. This is expected, and I could not find a meaningful improvement.

- The space complexity is O(n) due to the recursion stack, where n is the number of blank cells. I could not find a meaningful improvement.

- For maintainability, the program is well organized. The class names (Board, SudokuSolver, and PuzzleRunner) are clear, responsibilities are separated appropriately, and the comments explain the code thoroughly.

d(2): Copilots code:

```
/*
* Program: EECS 348 Assignment 4
* Description: C++ program that solves Sudoku puzzles using objects and
* recursive depth-first search with backtracking.
* Each solution must follow the row, column, and 3x3 block rules.
* The program finds and displays all solutions for each puzzle.
* Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt
* read from the current working directory.
* Digits 1 through 9 represent starting clues, and underscores
* represent empty cells.
* Outputs: Displays each puzzle filename, the original puzzle, and every
* solution found, or "No solution found" when no solution exists.
* Displays an error if a puzzle file cannot be opened, contains
* invalid characters, has an incorrect cell count, or cannot be read completely.
* Usage: ./Assignment4.exe
* Collaborators: None
* Other Sources: Claude (original C++ code selected as the basis for this assignment),
* Gemini (second GenAI program used for comparison),
* ChatGPT (assistance with prompting, analysis, testing,
* commenting, revisions, and source attribution)
* Author: Nathan Lenahan
* Creation Date: October 14, 2026
* Revision Date: October 15, 2026
* Revisions:
* - Added detailed comments and source attribution to the original Claude-generated code.
* - Revised load() to reject invalid characters, require exactly 81 cells,
* and check the entire input file while allowing whitespace.
* - Removed the unused get() method to simplify the Board class.
* - Replaced stored solution boards with immediate printing and a solution counter.
* - Added printSolution() to keep solution formatting separate from recursive search steps.
* - Revised run() to report the solution count after the search finishes.
* - Removed the vector include because completed solutions are no longer stored.
* - Revised findEmpty() and search() to resume after each filled cell,
* avoiding repeated scans of earlier positions during recursion.
* - Added specific loading-error descriptions for opening failures, invalid
```

```
* characters, extra content, incomplete puzzles, and file-reading failures.
*/

// The following include statements and namespace declaration are from the original code generated by Claude
#include <iostream> // Provides terminal output using cout and the ostream type
#include <fstream> // Provides file input using ifstream
#include <string> // Provides the string type and the to_string conversion
#include <cctype> // Provides isspace for checking whether a character is whitespace

using namespace std; // Allows standard library names to be used without the std:: prefix

// The following Board class is based on the original code generated by Claude
// Its load and findEmpty methods were revised and its unused get method was removed with help from ChatGPT
// It stores the Sudoku grid and handles loading, printing, and checking placements
class Board { // Starts the Board class
public: // Makes the following constants, constructor, and methods accessible outside the class

static const int SIZE = 9; // Defines the shared constant for the number of rows and columns
static const int BLOCK = 3; // Defines the shared constant for the number of rows and columns in each block

// The following Board constructor is from the original code generated by Claude
// It initializes every cell to 0 so a new board starts empty
Board() { // Starts the default Board constructor

for (int r = 0; r < SIZE; r++) // Loops through all nine rows
for (int c = 0; c < SIZE; c++) // Loops through all nine columns in the current row
cells[r][c] = 0; // Sets the current cell to 0 to represent an empty cell

} // Ends the Board constructor

// The following load method is based on the original code generated by Claude
// It was revised with help from ChatGPT to validate all cells and describe loading failures
// It accepts exactly 81 digits or underscores while allowing whitespace
bool load(const string& filename, string& errorMessage) { // Loads the puzzle and provides an error description through a reference

errorMessage.clear(); // Removes any error description left from an earlier call

ifstream in(filename.c_str()); // Attempts to open the puzzle file
```

```
if (!in) { // Checks whether the file could not be opened
errorMessage = "Could not open the file."; // Reports the failure without guessing its underlying cause
return false; // Stops loading because the file is unavailable
} // Ends the file-opening check

int count = 0; // Tracks the number of accepted cells
char ch; // Stores one character read from the file

while (in.get(ch)) { // Reads characters until the file ends or reading fails

if (isspace(static_cast<unsigned char>(ch))) { // Checks whether the character is whitespace
continue; // Skips whitespace without counting it as a cell
} // Ends the whitespace check

if (count >= SIZE * SIZE) { // Checks for non-whitespace content after all 81 cells
errorMessage = "Extra content found after 81 cells."; // Explains that the file contains too much cell
data
return false; // Rejects the extra content before accessing the board
} // Ends the extra-content check

int r = count / SIZE; // Calculates the row index for the next cell
int c = count % SIZE; // Calculates the column index for the next cell

if (ch >= '1' && ch <= '9') { // Checks whether the character is an allowed digit
cells[r][c] = ch - '0'; // Converts and stores the digit as an integer

} else if (ch == '_') { // Checks whether the character represents a blank
cells[r][c] = 0; // Stores 0 to represent the empty cell

} else { // Handles a character that is not an allowed digit or underscore
errorMessage = "Invalid character at cell " + to_string(count + 1)
+ ". Expected a digit from 1 through 9 or an underscore."; // Identifies the cell and explains the
allowed characters
return false; // Rejects the invalid input

} // Ends the cell-character validation

count++; // Increases the number of accepted cells

} // Ends the file-reading loop
```

```
if (in.bad() || !in.eof()) { // Checks for a reading failure rather than a normal end of the file
errorMessage = "A file-reading error occurred; the exact cause is unknown."; // Reports the stream failure without inventing a cause
return false; // Rejects a file that could not be read normally
} // Ends the reading-error check

if (count != SIZE * SIZE) { // Checks whether the file ended before all 81 cells were read
errorMessage = "Expected 81 cells, but found " + to_string(count) + "."; // Reports the actual number of accepted cells
return false; // Rejects the incomplete puzzle
} // Ends the cell-count check

return true; // Reports successful loading after every validation check passes

} // Ends the load method

// The following set method is from the original code generated by Claude
// It updates the value stored at a given row and column
void set(int r, int c, int v) { // Starts the method that changes one cell
cells[r][c] = v; // Stores the provided value in the specified cell
} // Ends the set method

// The following findEmpty method is based on the original code generated by Claude
// It was revised with help from ChatGPT to begin searching at a provided position
// This avoids repeatedly checking earlier cells that are already filled
bool findEmpty(int& row, int& col, int startIndex = 0) const { // Searches for an empty cell starting at the provided index

for (int position = startIndex; position < SIZE * SIZE; position++) { // Checks positions from the starting index through the last cell

int r = position / SIZE; // Converts the position into a row index
int c = position % SIZE; // Converts the position into a column index

if (cells[r][c] == 0) { // Checks whether the current cell is empty

row = r; // Provides the empty cell's row to the caller
col = c; // Provides the empty cell's column to the caller
return true; // Reports that an empty cell was found

} // Ends the empty-cell check
```

```
} // Ends the position-search loop

return false; // Reports that no empty cells were found at or after the starting index

} // Ends the findEmpty method

// The following canPlace method is from the original code generated by Claude
// It checks whether a digit already appears in the selected row, column, or block
bool canPlace(int row, int col, int v) const { // Starts checking whether value v can be placed at the
given position

for (int i = 0; i < SIZE; i++) { // Loops through all nine positions in the selected row and column
if (cells[row][i] == v) return false; // Rejects the value if it already appears in the row
if (cells[i][col] == v) return false; // Rejects the value if it already appears in the column
} // Ends the row and column checks

int br = (row / BLOCK) * BLOCK; // Calculates the starting row of the cell's 3x3 block
int bc = (col / BLOCK) * BLOCK; // Calculates the starting column of the cell's 3x3 block

for (int r = br; r < br + BLOCK; r++) // Loops through the three rows of the block
for (int c = bc; c < bc + BLOCK; c++) // Loops through the three columns of the block
if (cells[r][c] == v) return false; // Rejects the value if it already appears in the block
return true; // Reports that the value has no row, column, or block conflicts
} // Ends the canPlace method

// The following givensAreConsistent method is from the original code generated by Claude
// It checks each starting clue for conflicts while preserving the original board
bool givensAreConsistent() { // Starts checking whether the starting clues follow the Sudoku rules

for (int r = 0; r < SIZE; r++) // Loops through every row
for (int c = 0; c < SIZE; c++) { // Loops through every column in the current row

int v = cells[r][c]; // Saves the value currently stored in the cell

if (v == 0) continue; // Skips empty cells because they contain no clue to check

cells[r][c] = 0; // Temporarily removes the clue so it does not conflict with itself
bool ok = canPlace(r, c, v); // Checks the saved clue against the other digits in its row, column, and
block
cells[r][c] = v; // Restores the clue before continuing or returning
```

```
if (!ok) return false; // Rejects the board if the clue conflicts with another digit
} // Ends the column loop
return true; // Reports that all starting clues passed the conflict checks
} // Ends the givensAreConsistent method

// The following print method is from the original code generated by Claude
// It displays the board with underscores for blanks and dividers between blocks
void print(ostream& out) const { // Starts printing to the provided output stream without changing the board

for (int r = 0; r < SIZE; r++) { // Loops through each row of the board
if (r > 0 && r % BLOCK == 0) out << "------+-------+------\n"; // Prints a horizontal divider before row indexes 3 and 6
for (int c = 0; c < SIZE; c++) { // Loops through each column in the current row
if (c > 0 && c % BLOCK == 0) out << "| "; // Prints a vertical divider before column indexes 3 and 6
if (cells[r][c] == 0) out << "_"; // Displays an underscore for an empty cell
else out << cells[r][c]; // Displays the digit stored in a filled cell
out << (c < SIZE - 1 ? " " : ""); // Adds a space after each cell except the last cell in the row
} // Ends the column loop

out << "\n"; // Moves to the next line after printing the row

} // Ends the row loop
} // Ends the print method
// Added with help from Copilot
// Writes the board in the same format as the puzzle input files
void writePuzzleFormat(ostream& out) const {

for (int r = 0; r < SIZE; r++) {

for (int c = 0; c < SIZE; c++) {

if (cells[r][c] == 0)
out << '_';
else
out << cells[r][c];

}

out << "\n";
}
```

```
}
private: // Prevents code outside Board from directly accessing the following data
int cells[SIZE][SIZE]; // Stores the 9x9 grid as integers, with 0 representing an empty cell
}; // Ends the Board class

// The following SudokuSolver class is based on the original code generated by Claude
// Solution handling and search positioning were revised with help from ChatGPT
// This class finds all solutions using recursive backtracking
class SudokuSolver { // Starts the SudokuSolver class
public: // Makes the constructor and solveAll method accessible outside the class

// The following constructor is based on the original code generated by Claude
// It was revised with help from ChatGPT to initialize a solution counter
explicit SudokuSolver(const Board& puzzle)
: board(puzzle), solutionCount(0) {} // Copies the puzzle and initializes the number of solutions to 0

// The following solveAll method is based on the original code generated by Claude
// It was revised with help from ChatGPT to accept an output stream and return a count instead of a
solution vector
unsigned long long solveAll(ostream& out) { // Starts solving and returns the number of solutions
found

solutionCount = 0; // Resets the counter before starting a new search

if (board.givensAreConsistent()) { // Allows searching only when the starting clues have no conflicts
search(out); // Starts searching from position 0 using the default starting index
} // Ends the starting-clue check

return solutionCount; // Returns the total number of solutions found

} // Ends the solveAll method

private: // Restricts access to the working board, counter, and helper methods

Board board; // Stores the working board modified during backtracking
unsigned long long solutionCount; // Counts solutions without storing their completed boards

// The following printSolution method was added with help from Copilot
// It keeps solution formatting in a separate method from the recursive search
void printSolution(ostream& out) const {
board.writePuzzleFormat(out);
out << "\n";
```

```
}

// The following search method is based on the original code generated by Claude
// It was revised with help from ChatGPT to print solutions immediately and resume searching after each filled cell
// Earlier positions are already filled along the current recursive path
void search(ostream& out, int startIndex = 0) { // Starts a recursive search using the provided output stream and cell position

int row, col; // Receives the coordinates of the next empty cell

if (!board.findEmpty(row, col, startIndex)) { // Checks whether no empty cells remain from the starting position onward

solutionCount++; // Increases the count because all cells are filled along this search path
printSolution(out); // Prints the completed board before backtracking changes it
return; // Ends this call while allowing earlier calls to continue searching

} // Ends the completed-board check

int nextIndex = row * Board::SIZE + col + 1; // Calculates the position immediately after the selected cell

for (int v = 1; v <= 9; v++) { // Tries candidate digits from 1 through 9

if (board.canPlace(row, col, v)) { // Checks whether the candidate has no row, column, or block conflicts

board.set(row, col, v); // Places the candidate in the selected empty cell
search(out, nextIndex); // Recursively searches beginning after the cell just filled
board.set(row, col, 0); // Clears the attempted placement before trying another candidate

} // Ends the allowed-placement check

} // Ends the candidate loop

} // Ends the search method

}; // Ends the SudokuSolver class

// The following PuzzleRunner class is based on the original code generated by Claude
```

```
// Its run method was revised with help from ChatGPT
// It loads one puzzle file, runs the solver, and displays the original puzzle and results
class PuzzleRunner { // Starts the PuzzleRunner class
public: // Makes the following constructor and method accessible outside the class

// The following PuzzleRunner constructor is from the original code generated by Claude
// It stores the name of the puzzle file this object will process
explicit PuzzleRunner(const string& file) : filename(file) {} // Copies the filename into the member and prevents implicit conversion to PuzzleRunner

// The following run method is based on the original code generated by Claude
// It was revised with help from ChatGPT to display specific loading errors and use solutions printed during the search
// It displays the original puzzle and reports the final solution count or no-solution message
void run() {
cout << "==============================\n";
cout << "Puzzle file: " << filename << "\n";
cout << "==============================\n";

Board puzzle;
string errorMessage;

if (!puzzle.load(filename, errorMessage)) {
cout << "Error loading "
<< filename
<< ": "
<< errorMessage
<< "\n\n";
return;
}

size_t startDigit =
filename.find_first_of("123456789");

string solutionFilename = "solution";

if (startDigit != string::npos)
solutionFilename += filename[startDigit];

solutionFilename += ".txt";

ofstream out(solutionFilename.c_str());
```

```
if (!out) {
cout << "Could not create "
<< solutionFilename
<< "\n\n";
return;
}

SudokuSolver solver(puzzle);

unsigned long long solutionCount =
solver.solveAll(out);

if (solutionCount == 0) {

out << "No solution found.\n";

} else {

out << "Solutions found: "
<< solutionCount
<< "\n";

}

out.close();

cout << "Results written to "
<< solutionFilename
<< "\n\n";
}

private: // Restricts direct access to the following data member to PuzzleRunner
string filename; // Stores the name of the puzzle file this runner processes

}; // Ends the PuzzleRunner class

// The following main function is from the original code generated by Claude
// It creates and runs a PuzzleRunner object for each of the five puzzle files
int main() { // Starts the program's main function
for (int i = 1; i <= 5; i++) { // Loops through puzzle numbers 1 through 5
```

```
PuzzleRunner runner("puzzle" + to_string(i) + ".txt"); // Builds the filename and creates an object to process it
runner.run(); // Loads the puzzle, displays it, solves it, and displays the results
} // Ends the loop that processes the five puzzle files

return 0; // Ends the program successfully

} // Ends the main function
```

e.

- Copilots code is correct for all Sudoku inputs that meet the stated requirements. It validates input files, checks that the starting clues are consistent, and uses recursive backtracking to find all solutions.

- The worst-case time complexity is $O(9^n)$, where n is the number of blank cells. This is expected, and I could not find an improvement while still finding all solutions.

- The space complexity is O(n) due to the recursion stack, where n is the number of blank cells (the board itself is a fixed 9x9). I could not find an improvement.

- For maintainability, the program is well organized. The class names (Board, SudokuSolver, and PuzzleRunner) are clear, responsibilities are separated appropriately, and the comments explain the code thoroughly. The later additions (writePuzzleFormat and the solution-file handling in run) are not commented as consistently as the rest.

f. I chose Claudes code. It uses cleaner input (accepts an int instead of string parsing). It also gives more data after use.

g. I reformatted the file, Improved givensAreConsistent(), and improved PuzzleRunner::run(). More info is available in the prologue comments.